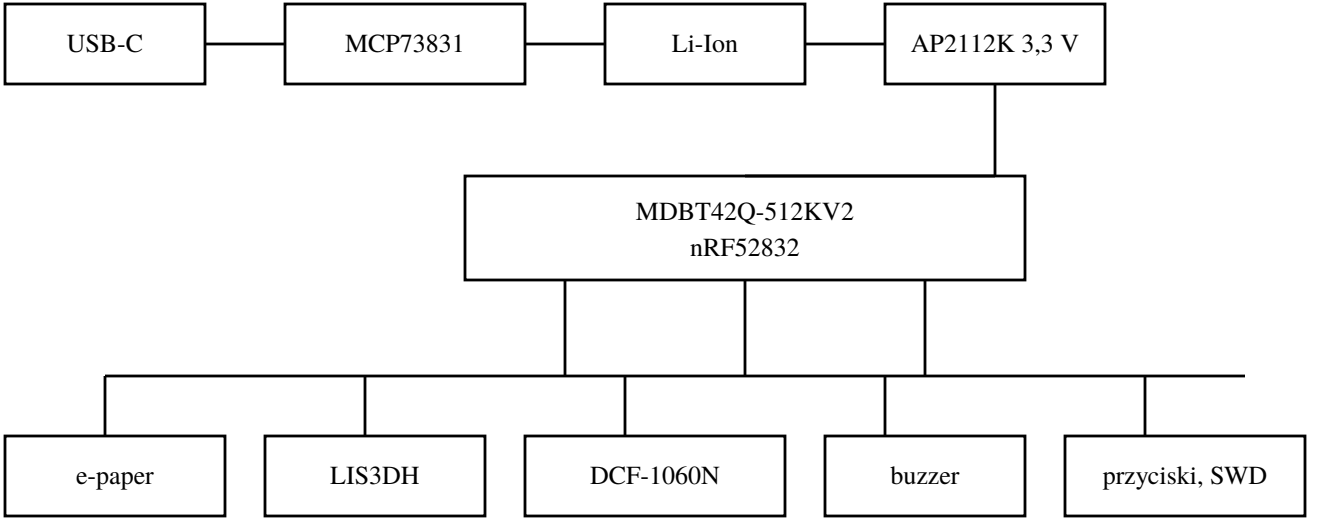
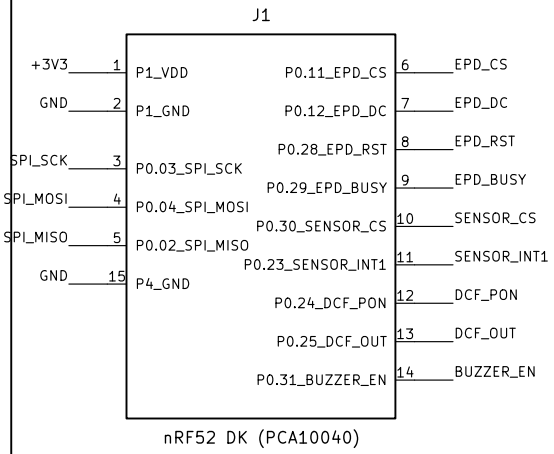
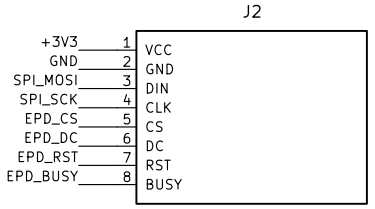

USB-C
MCP73831
Li-Ion
AP2112K 3,3 V
MDBT42Q-512KV2
nRF52832
e-paper
LIS3DH
DCF-1060N
buzzer
przyciski, SWD

BLOCK 1: HOST CONTROLLER INTERFACE (nRF52 DK / PCA10040)

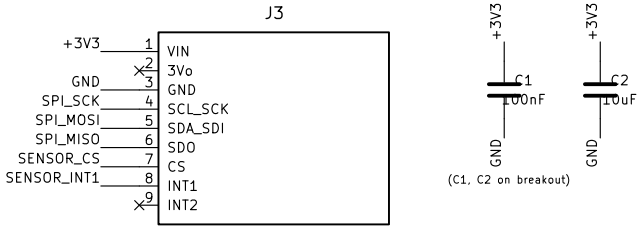


BLOCK 2: DISPLAY INTERFACE (Waveshare 2.13in HAT)

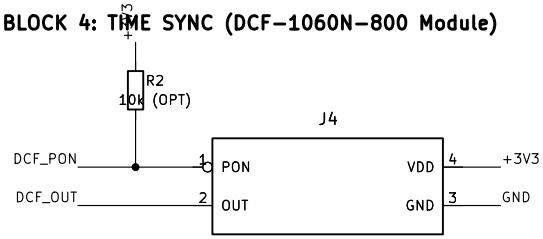


Verify BS1 = 0 (4-wire SPI) on HAT rear switch

BLOCK 3: MOTION SENSOR (Adafruit LIS3DH #2809)



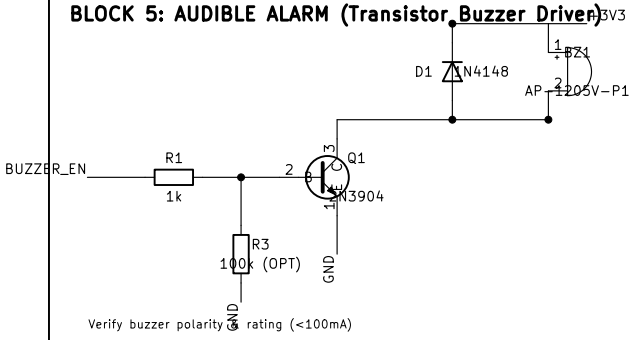
BLOCK 4: TIME SYNC (DCF-1060N-800 Module)



PON = 0V: Enabled (~68 uA) | PON = 3.3V: Disabled (0.0 uA)

MEASURE OUT HIGH LEVEL BEFORE CONNECTING DK P0.25!

BLOCK 5: AUDIBLE ALARM (Transistor Buzzer Driver)



Verify buzzer polarity & rating (<100mA)

PROTOTYPE SAFETY & INTEGRATION RULES

1. POWER: Feed the 3V3 rail only from DK header P1 pin VDD (USB powered). Never use 5V, External supply, VDD_nRF or P22. NEVER connect KORAD bench supply and the 3V3 rail together!
2. SAFE CONTROL STATES: Set DCF_PON HIGH (disabled, 0.0uA). Set EPD_CS and SENSOR_CS HIGH (unselected). Set BUZZER_EN LOW (off).
3. DCF-77 VERIFICATION: Measure DCF OUT high-level voltage before connecting to DK P0.25. Orient antenna perpendicular to Frankfurt.
4. E-PAPER CONFIGURATION: Verify switch BS1 = 0 on back of Waveshare HAT for 4-wire SPI mode before powering display rail.
5. BUZZER DRIVER: Verify 2N3904 TO-92 pinout (1=Emitter, 2=Base, 3=Collector). Diode D1 cathode connects to +3V3 to clamp flyback spikes.
6. STAGED BRING-UP: 1) DK Blinky -> 2) E-Paper Demo -> 3) LIS3DH WHO_AM_I -> 4) DCF Standalone -> 5) DCF Sync -> 6) Buzzer Alarm.

nRF52 DK (PCA10040) + E-Paper + LIS3DH + DCF-77 + Buzzer Driver

Bachelor Thesis (APD)

Sheet: /

File: breadboard-prototype.kicad_sch

Title: Ultra-Low-Power Bedside Clock – Breadboard Prototype

Size: A4	Date: 2026-09-13	**Rev: v1.1**
KiCad E.D.A. 10.0.4		Id: 1/1

Procesor (CPU)

Start

stan bezpieczny pinów, SoftDevice,
LFXO i RTC1, rozkład z flash,
pierwsze odświeżenie wyświetlacza e-Paper

Odświeżanie wyświetlacza e-Paper

SPI do panelu, między krokami
sen WFE do BUSY lub terminu,
potem głęboki sen panelu

panel w głębokim śnie

żądanie odświeżenia
i okno DCF zamknięte

Praca CPU

app_poll(): UART, zegar, alarm,
DCF, BLE, ustawienie terminu RTC1

brak pracy;
RTC1 CC0 = najbliższy termin

przerwanie budzące

Sen System ON (WFE)

RTC1 i LFXO pracują,
SPIM, PWM, UARTE wyłączone

Przerwania budzące CPU:
RTC1 COMPARE0 (termin aplikacji)
RTC1 COMPARE1 (oczekiwanie sterownika)
RTC1 OVERFLOW (co 512 s)
GPIOTE PORT: przyciski (zawsze)
GPIOTE PORT: DCF_OUT (tylko okno DCF)
GPIOTE IN: INT1 LIS3DH (tylko alarm)
GPIOTE IN: EPD_BUSY (tylko odświeżanie)
SWI2: zdarzenia SoftDevice (BLE, flash)
UARTE0: konsola (tylko wersja z UART)

Tryby zasilające dodatkowe układy (niezależne od snu CPU)

Odbiornik DCF wyłączony

start, 03:00 lub
ponowienie po 1 h;
tylko bez alarmu

dwie zgodne ramki
lub upływ 5 min

Okno DCF

PON niski, odbiornik zasilany,
zbocza DCF_OUT, maks. 5 min

Cisza

minuta zgodna
z rozkładem
(też po ustawieniu czasu)

przycisk,
20 ruchów w 30 s
lub upływ 60 s

Alarm

brzęczyk PWM 1 s / 1 s,
LIS3DH aktywny,
INT1 włączone

Radio wyłączone

przycisk
trzymany 2 s,
brak alarmu

upływ 60 s
lub alarm

Rozgłaszanie BLE

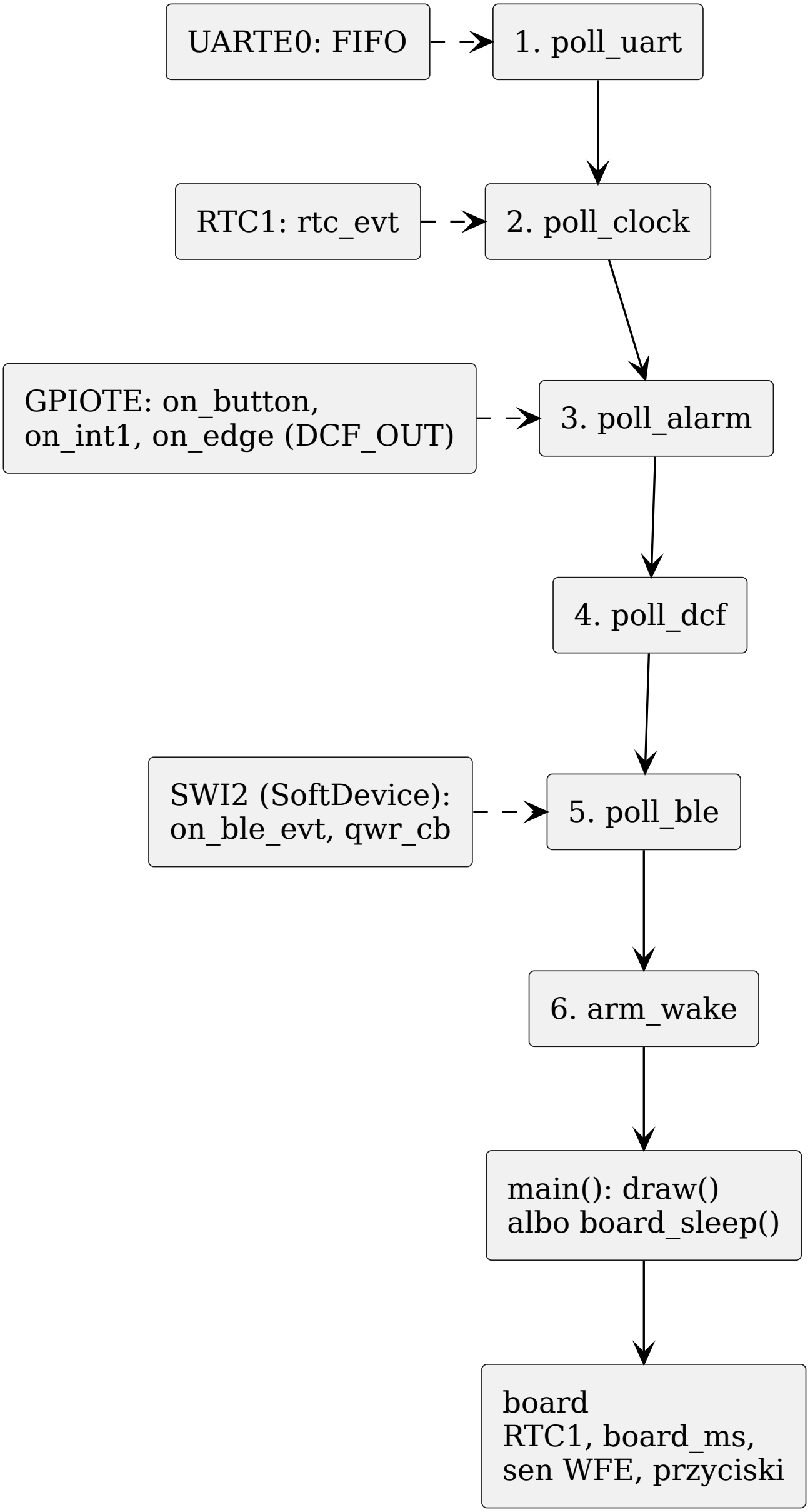
co 100 ms, maks. 60 s

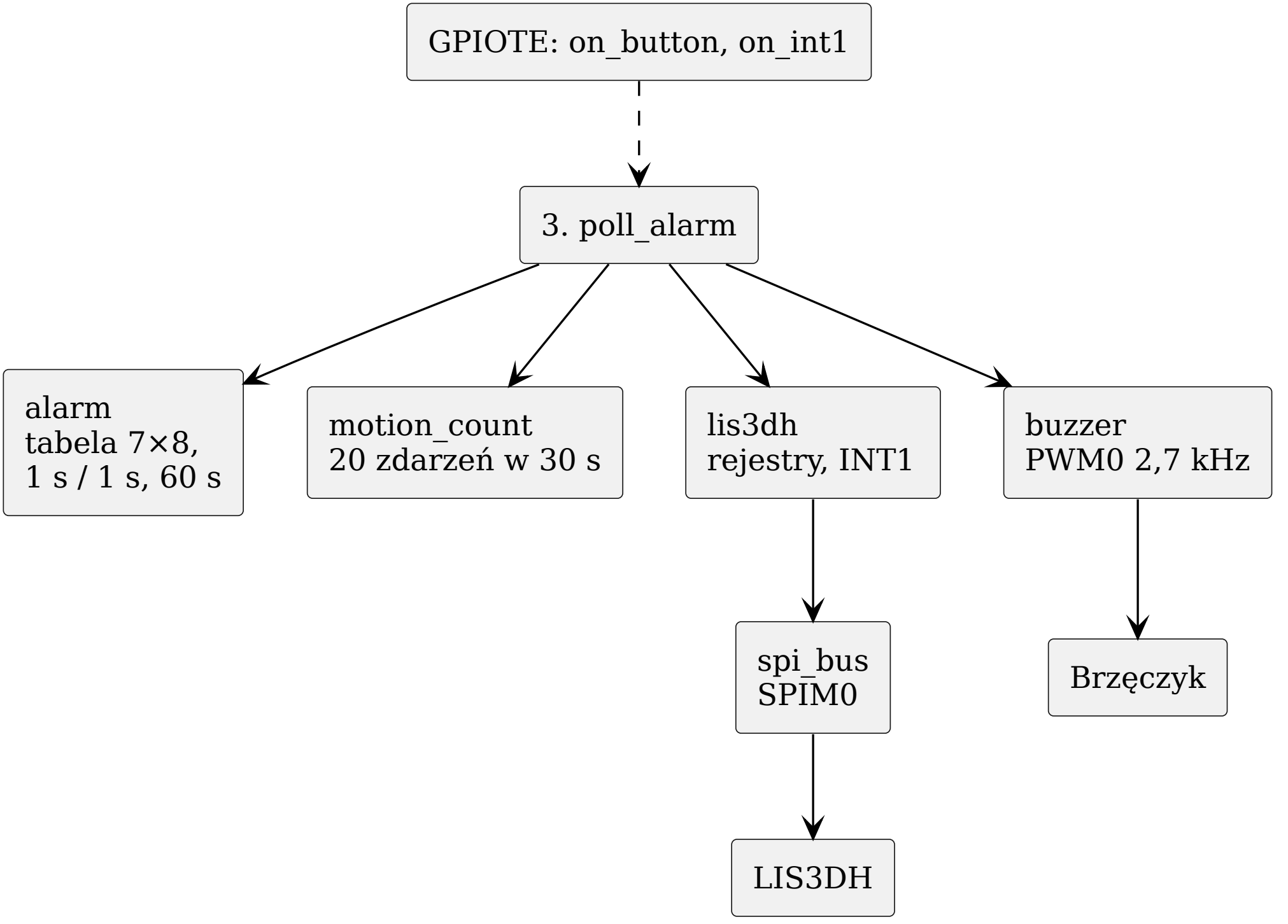
połączenie
z przeglądarki

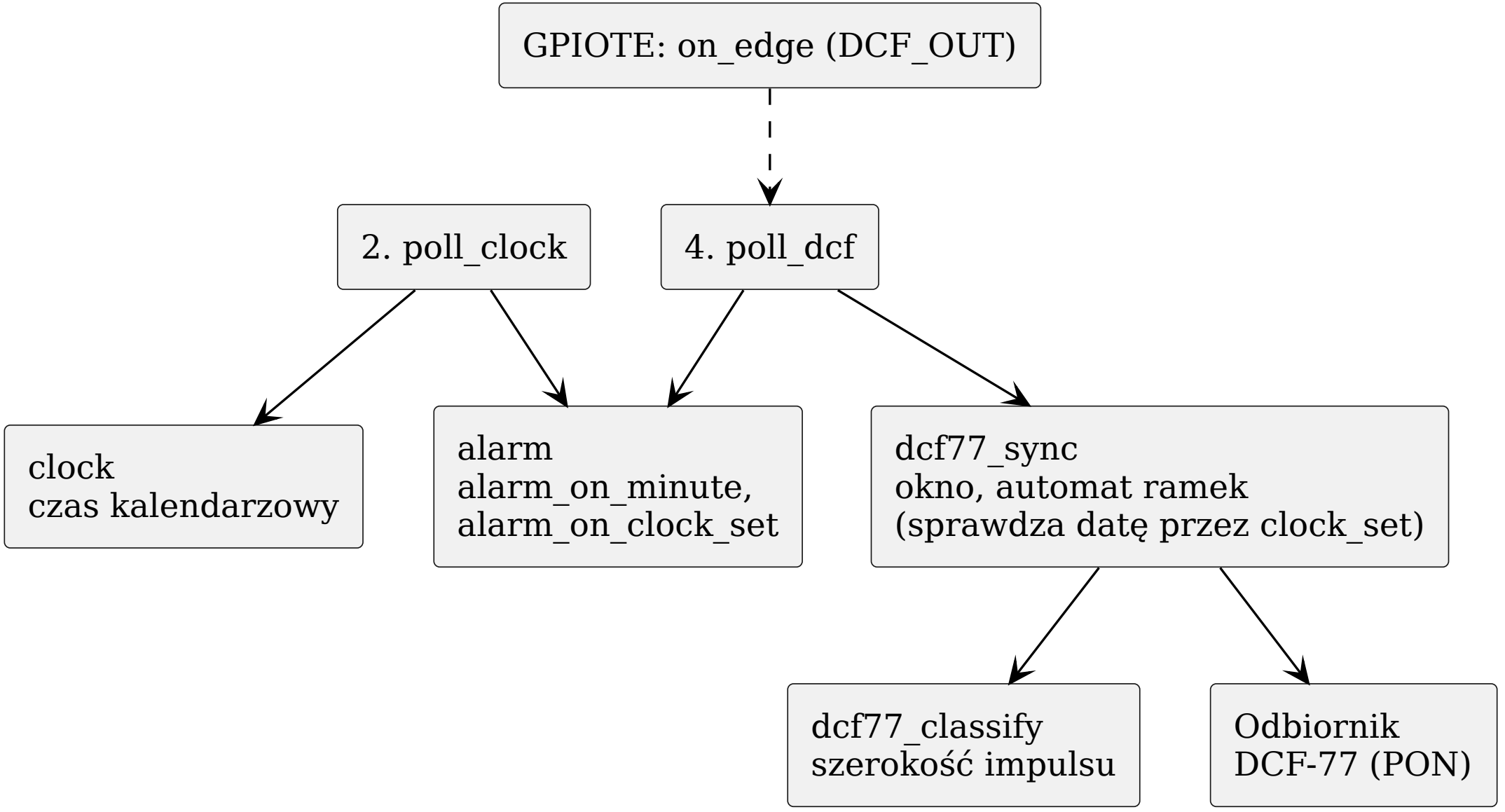
rozłączenie, alarm,
60 s bez zapisu
lub 30 s po zapisie

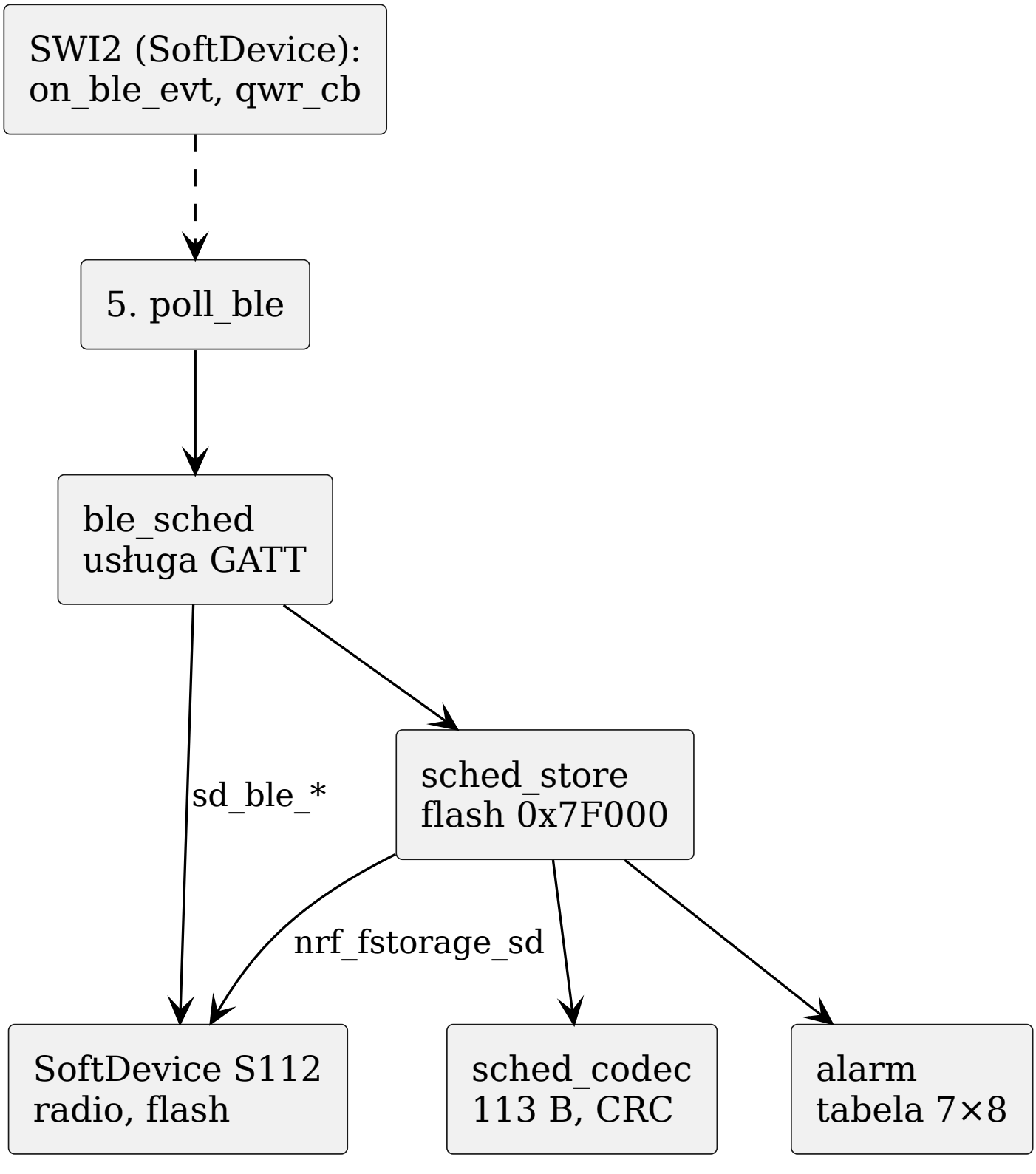
Połączenie BLE

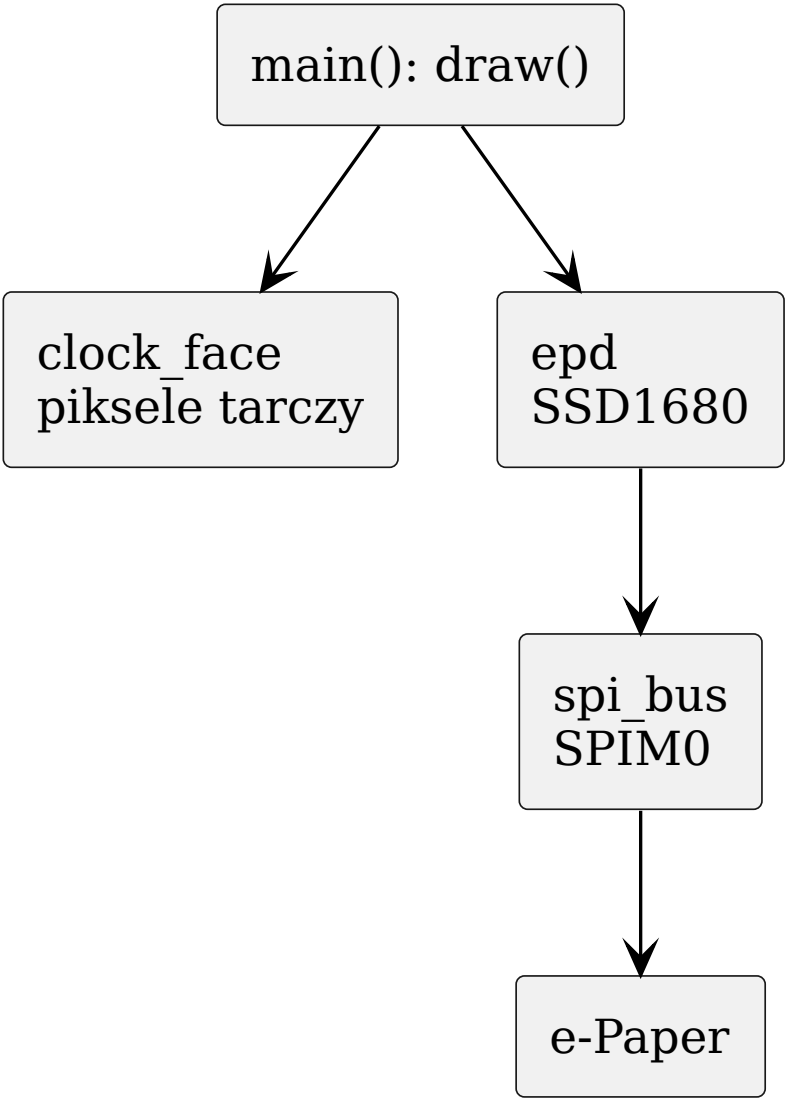
Alarm ma pierwszeństwo: zamyka radio BLE i wstrzymuje otwarcie okna DCF
(okno porzucone, gdy alarm trwa dłużej niż 5 min).
Okno DCF wstrzymuje odświeżanie wyświetlacza e-Paper do swojego końca.


UARTE0: FIFO
1. poll_uart
RTC1: rtc_evt
2. poll_clock
GPIOTE: on_button,
on_int1, on_edge (DCF_OUT)
3. poll_alarm
4. poll_dcf
SWI2 (SoftDevice):
on_ble_evt, qwr_cb
5. poll_ble
6. arm_wake
main(): draw()
albo board_sleep()
board
RTC1, board_ms,
sen WFE, przyciski

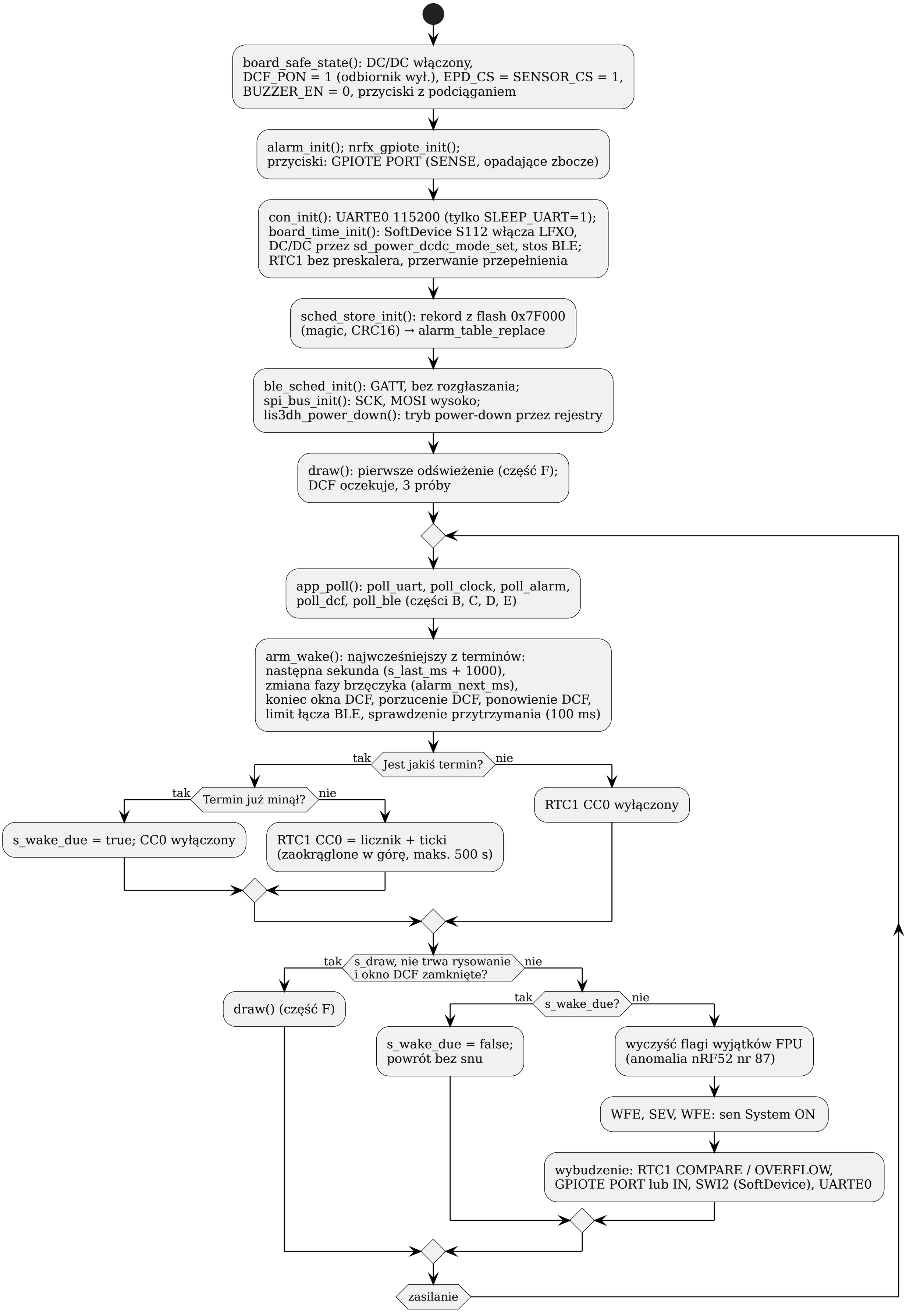

GPIOTE: on_button, on_int1
3. poll_alarm
alarm
tabela 7×8,
1 s / 1 s, 60 s
motion_count
20 zdarzeń w 30 s
lis3dh
rejestry, INT1
buzzer
PWM0 2,7 kHz
spi_bus
SPIM0
Brzęczyk
LIS3DH


GPIOTE: on_edge (DCF_OUT)
2. poll_clock
4. poll_dcf
clock
czas kalendarzowy
alarm
alarm_on_minute,
alarm_on_clock_set
dcf77_sync
okno, automat ramek
(sprawdza datę przez clock_set)
dcf77_classify
szerokość impulsu
Odbiornik
DCF-77 (PON)

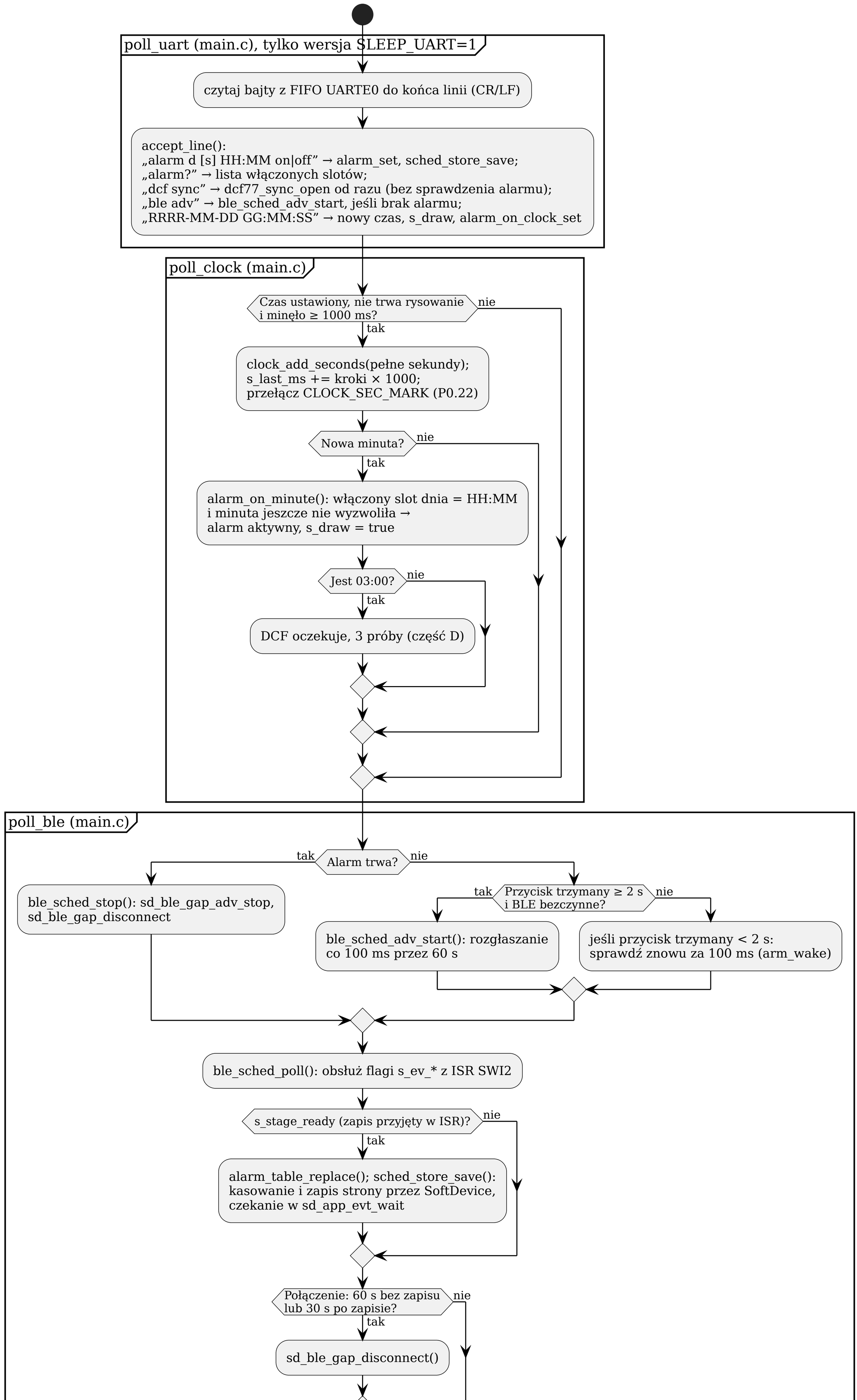

SWI2 (SoftDevice):
on_ble_evt, qwr_cb
5. poll_ble
ble_sched
usługa GATT
sd_ble_*
sched_store
flash 0x7F000
nrf_fstorage_sd
SoftDevice S112
radio, flash
sched_codec
113 B, CRC
alarm
tabela 7×8


main(): draw()
clock_face
piksele tarczy
epd
SSD1680
spi_bus
SPIM0
e-Paper

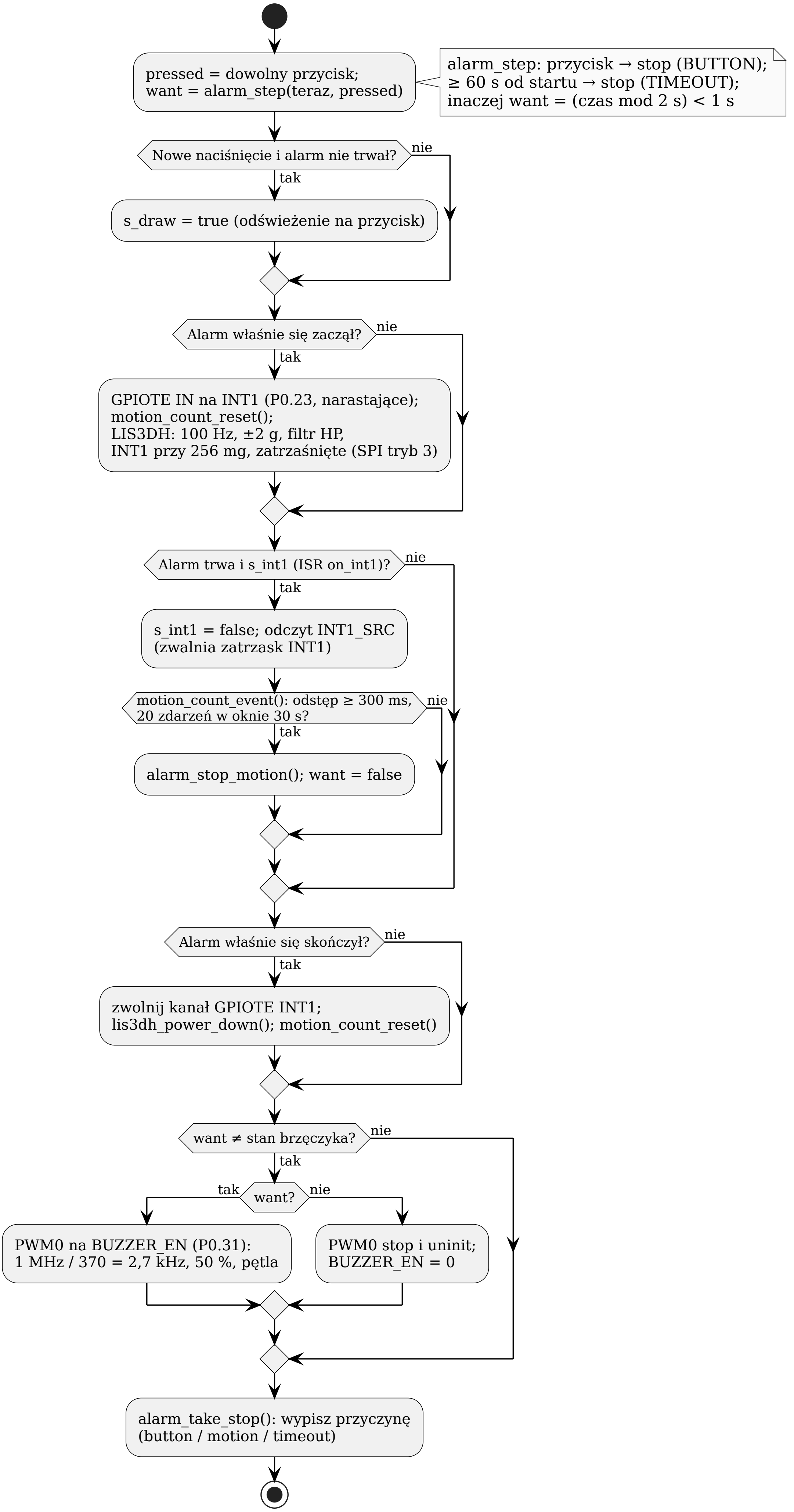
L3-A: start, pętla główna, termin RTC1 i sen



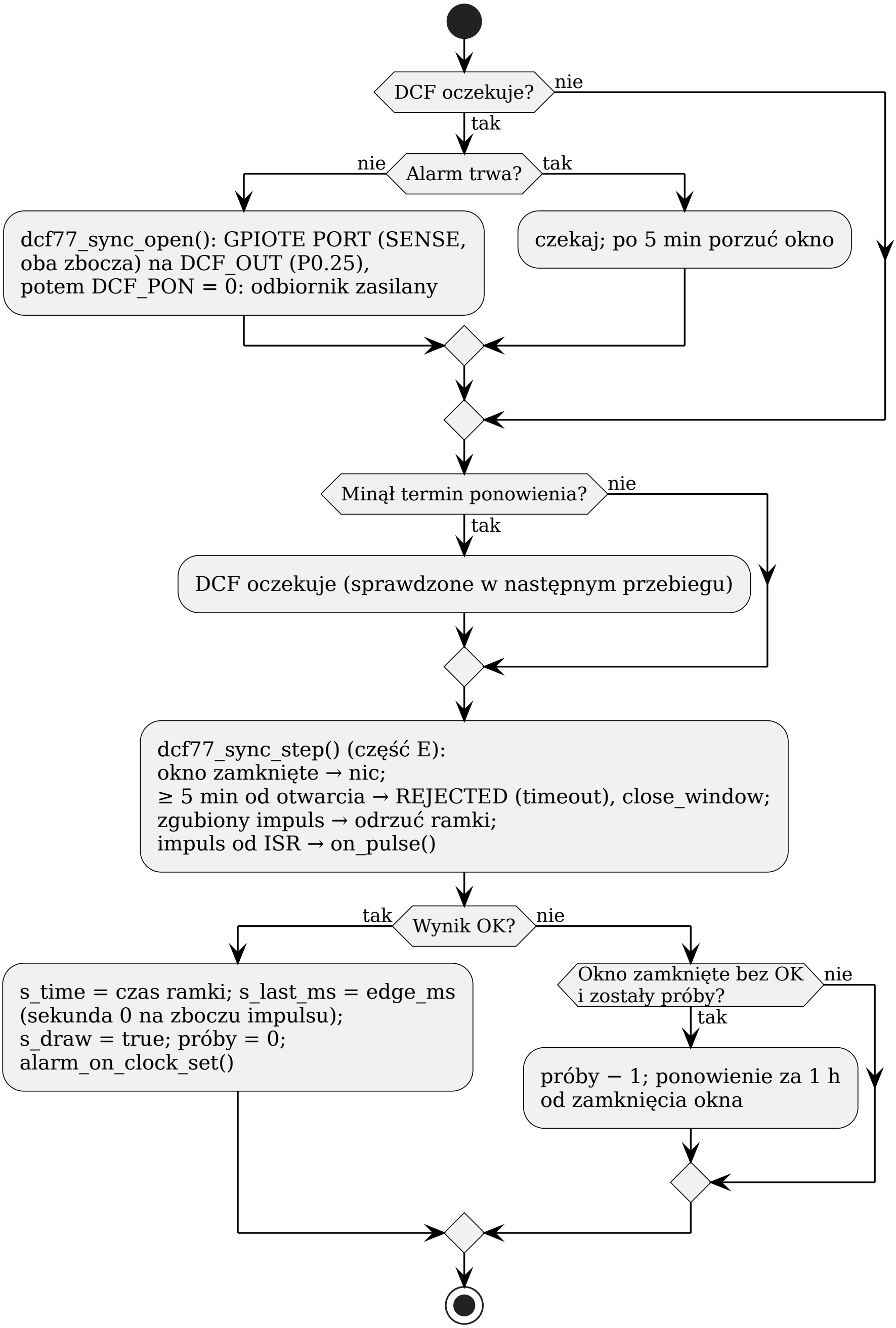
L3-B: konsola, zegar i BLE (kroki 1, 2 i 5 app_poll)



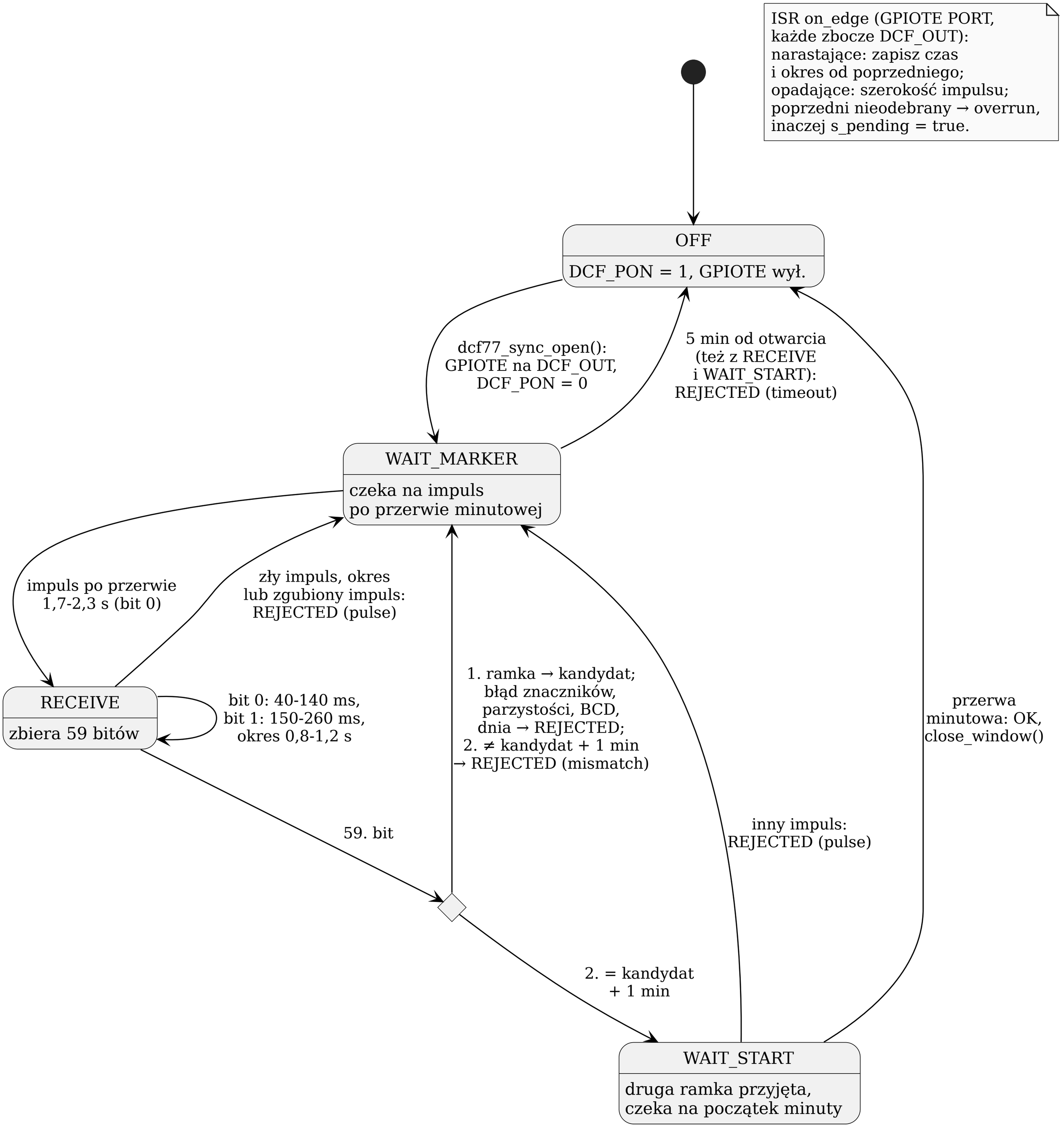
L3-C: alarm, ruch i brzęczyk (krok 3 app_poll)



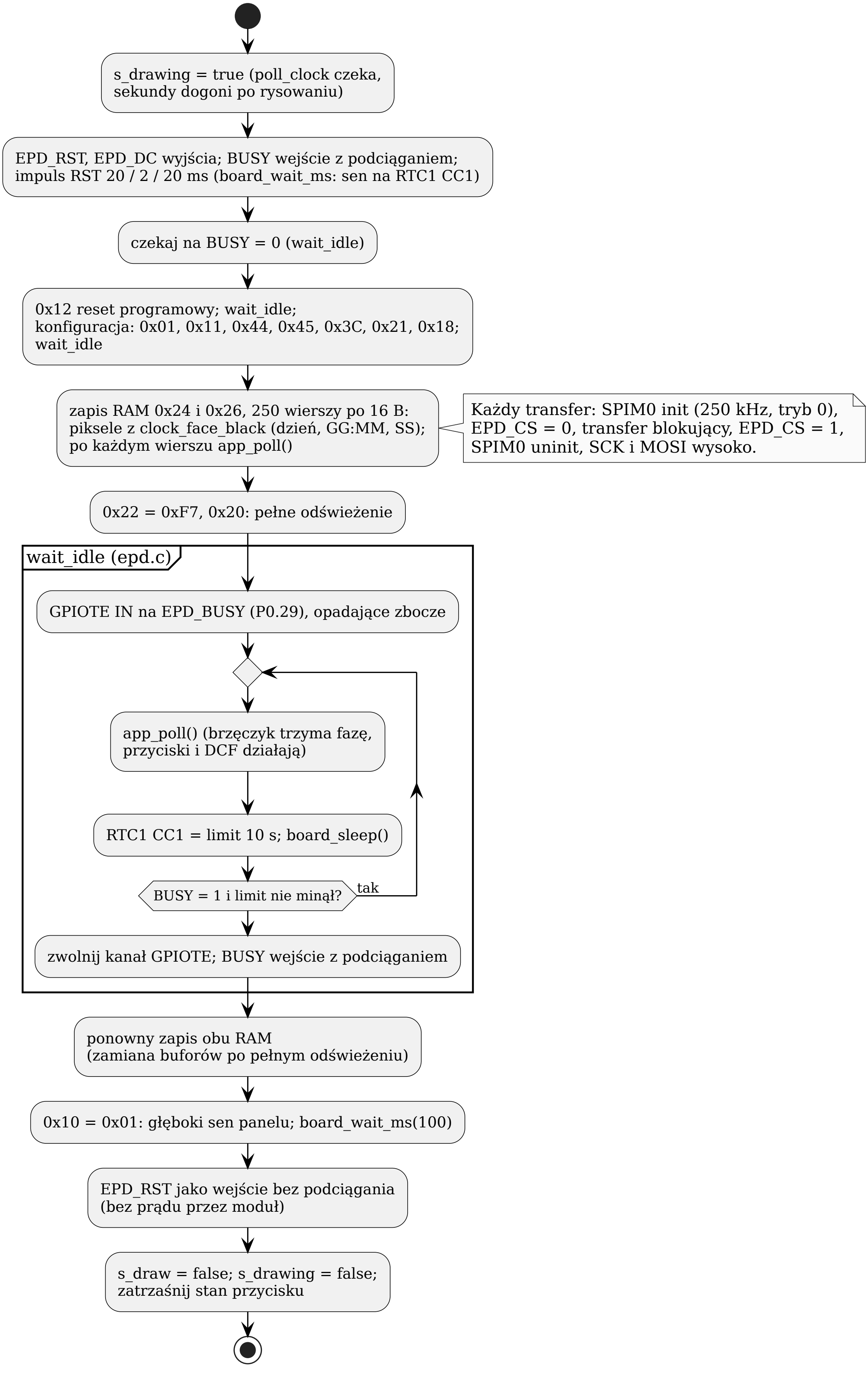
L3-D: okno DCF-77 (krok 4 app_poll)

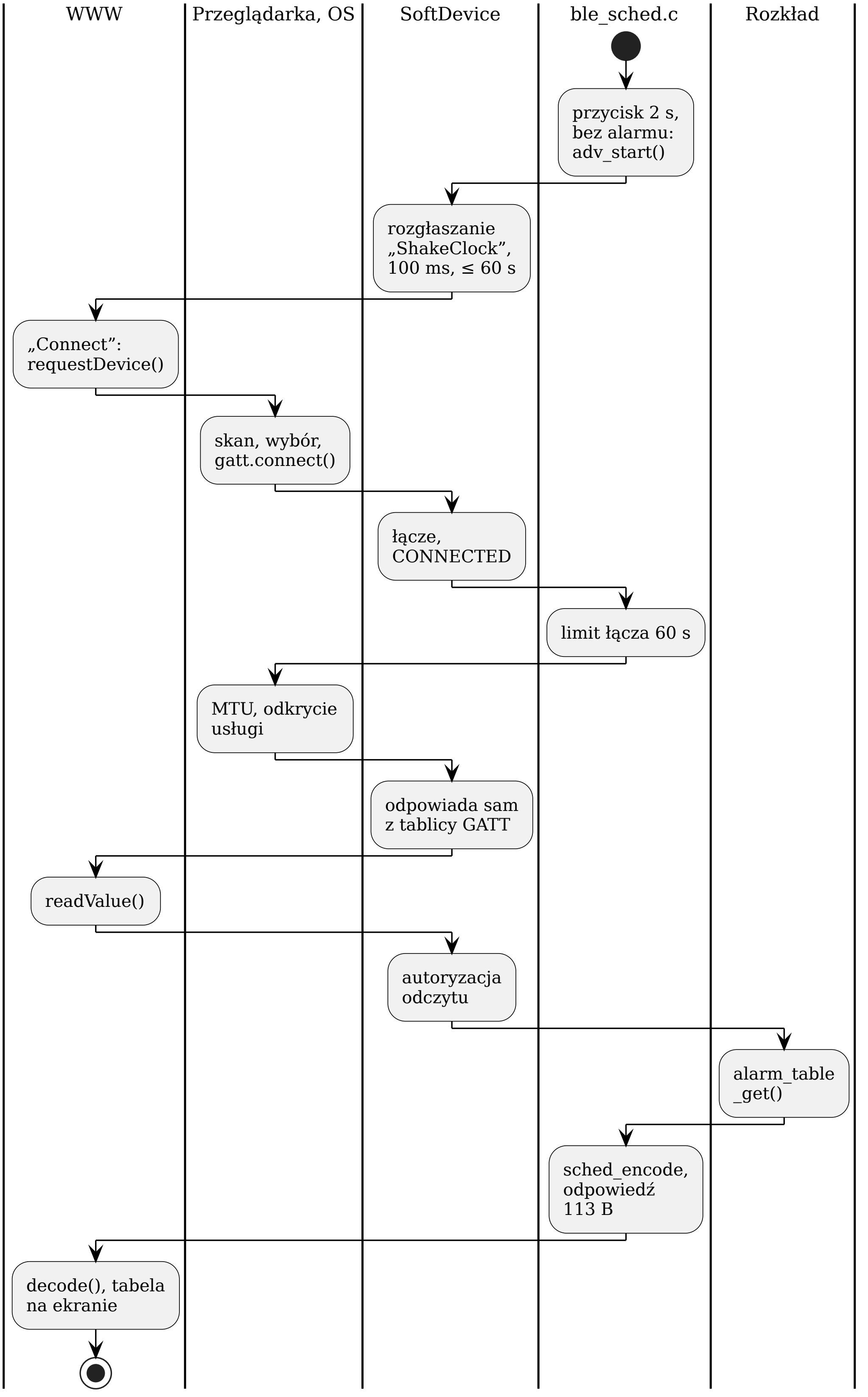


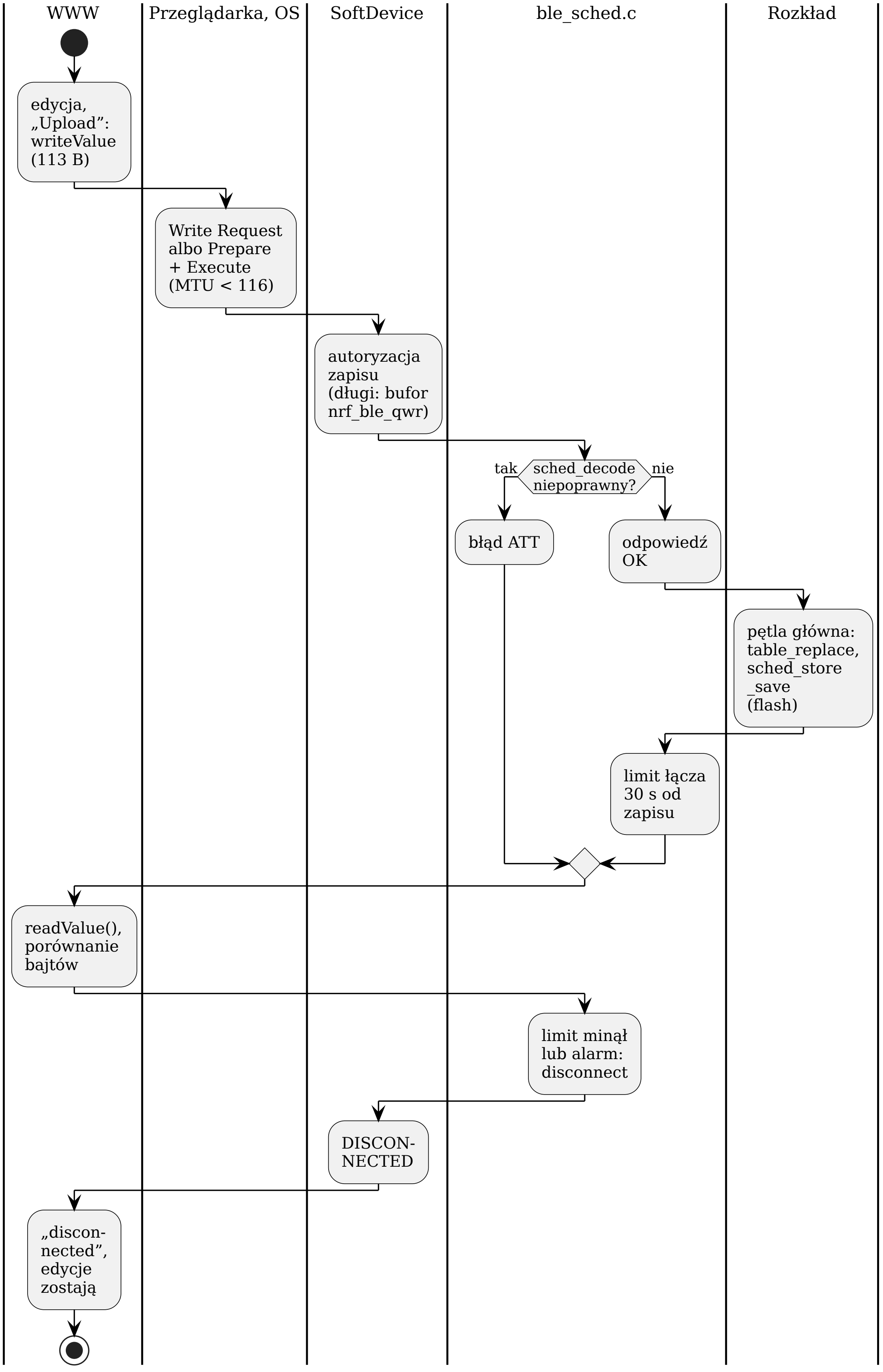
L3-E: dekoder DCF-77 (dcf77_sync_step, on_pulse)

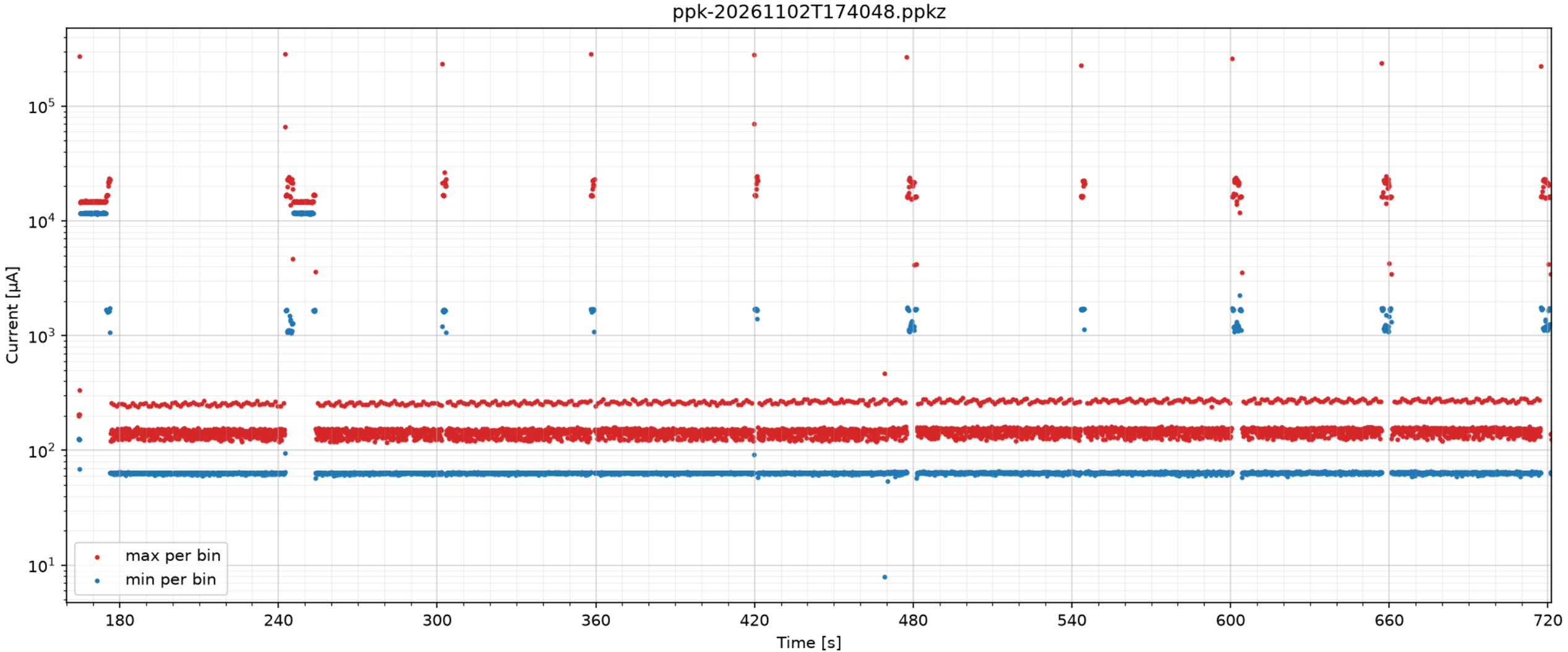


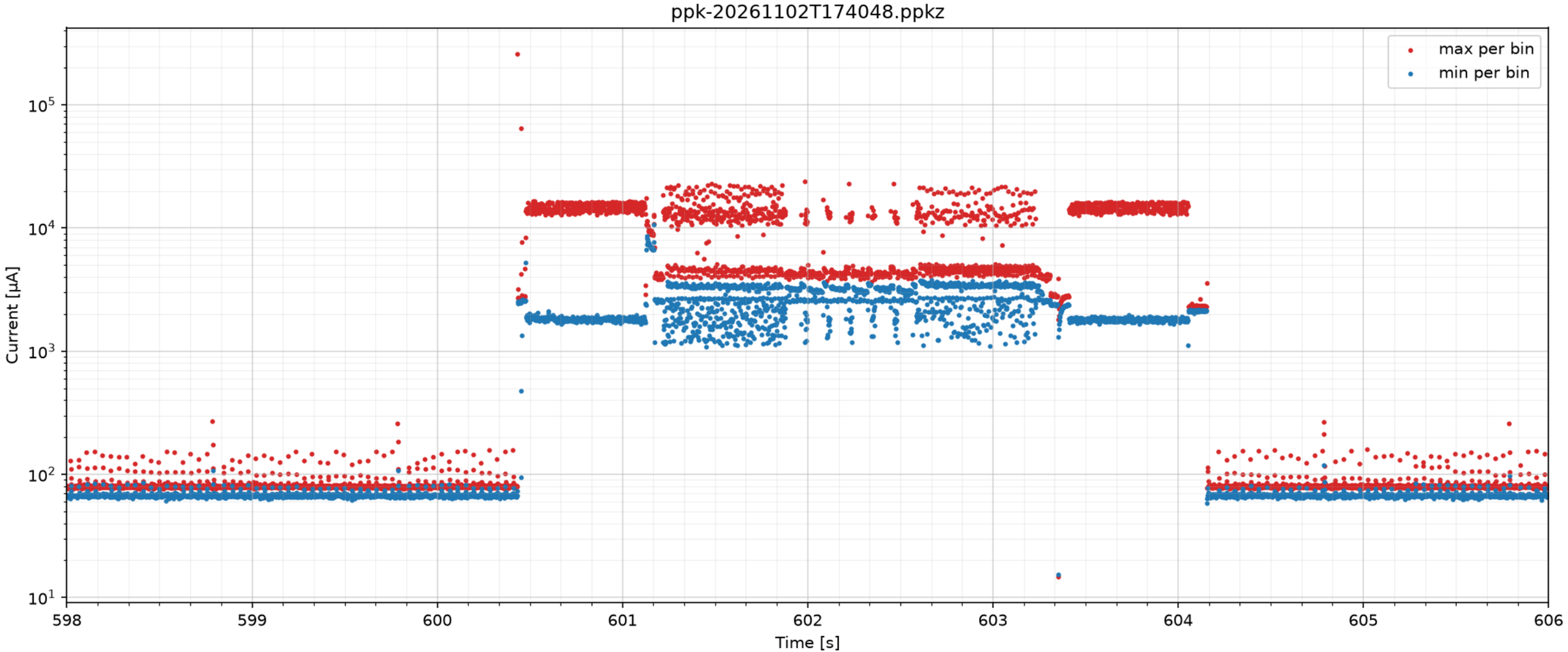
L3-F: odświeżenie wyświetlacza e-Paper (draw)




WWW
Przeglądarka, OS
SoftDevice
ble_sched.c
Rozkład
przycisk 2 s,
bez alarmu:
adv_start()
rozgłaszanie
„ShakeClock”,
100 ms, ≤ 60 s
„Connect”:
requestDevice()
skan, wybór,
gatt.connect()
łącze,
CONNECTED
limit łącza 60 s
MTU, odkrycie
usługi
odpowiada sam
z tablicy GATT
readValue()
autoryzacja
odczytu
alarm_table
_get()
sched_encode,
odpowiedź
113 B
decode(), tabela
na ekranie


WWW
Przeglądarka, OS
SoftDevice
ble_sched.c
Rozkład
edycja, „Upload”: writeValue (113 B)
Write Request albo Prepare + Execute (MTU < 116)
autoryzacja zapisu (długi: bufor nrf_ble_qwr)
tak
sched_decode niepoprawny?
nie
błąd ATT
odpowiedź OK
pętla główna: table_replace, sched_store _save (flash)
limit łącza 30 s od zapisu
readValue(), porównanie bajtów
limit minął lub alarm: disconnect
DISCON-NECTED
„discon-nected”, edycje zostają


ppk-20261102T174048.ppkz
Current [µA]
10^5
10^4
10^3
10^2
10^1
max per bin
min per bin
180
240
300
360
420
480
540
600
660
720
Time [s]


ppk-20261102T174048.ppkz
max per bin
min per bin
Current [µA]
10^5
10^4
10^3
10^2
10^1
598
599
600
601
602
603
604
605
606
Time [s]

Strona WWW
Przeglądarka i BLE OS
SoftDevice S112
ble_sched.c (SWI2)
pętla główna
alarm.c

Rozgłaszanie

1 przycisk 2 s, brak alarmu: adv_start (100 ms, 60 s)

2 ADV_IND „ShakeClock”, SCAN_RSP z UUID

Połączenie

3 wybór, gatt.connect()

4 CONNECT_IND

5 GAP_EVT_CONNECTED → s_ev_connect

6 wymiana MTU (do 247)

7 ble_sched_poll: limit 60 s

8 odkrycie usługi i charakterystyki (SoftDevice sam)

Odczyt rozkładu

9 readValue(): ATT Read Request

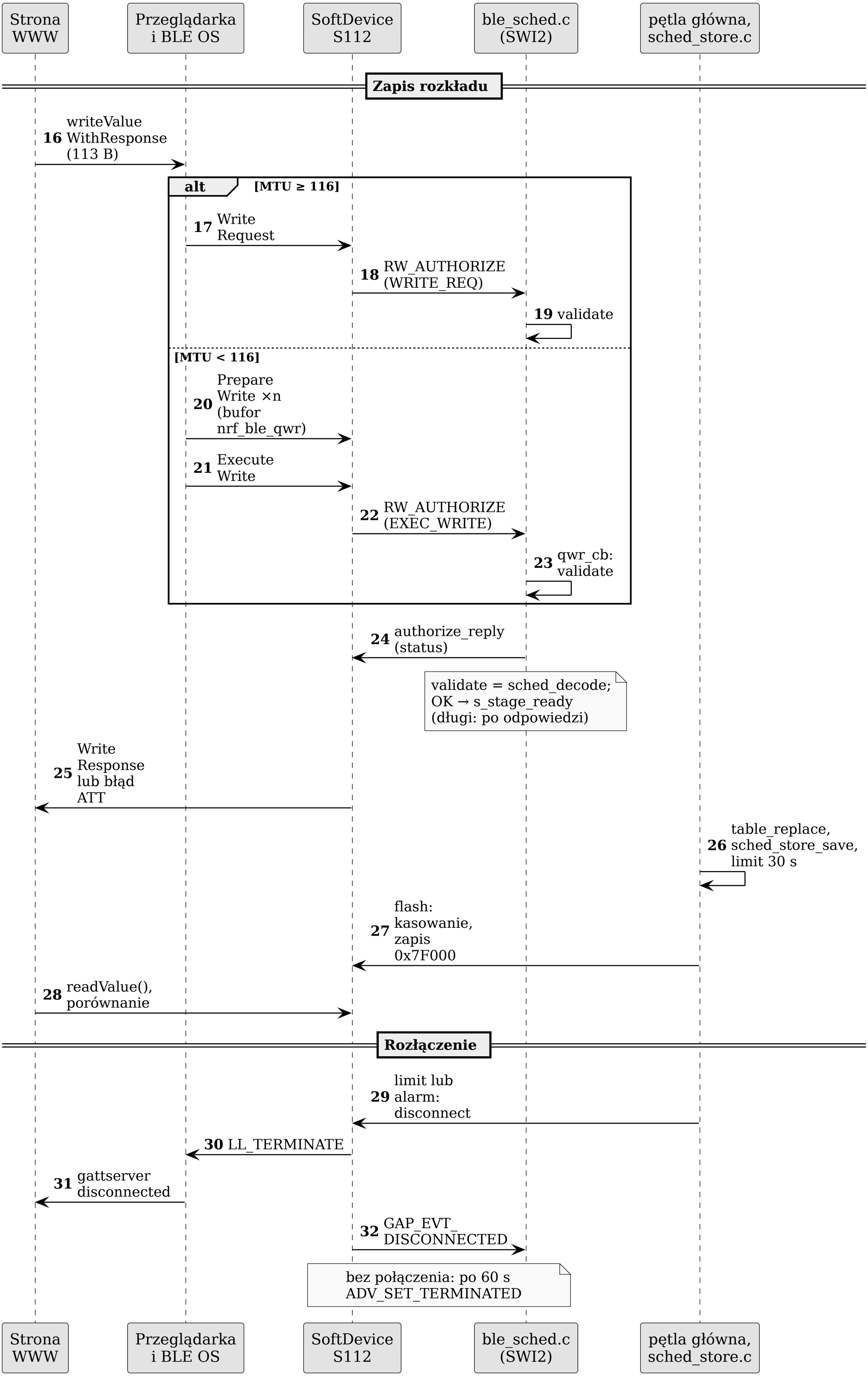
10 RW_AUTHORIZE_REQUEST (odczyt)

11 alarm_table_get()

12 sched_encode, authorize_reply (113 B)

13 ATT Read Response → decode()

Strona WWW
Przeglądarka i BLE OS
SoftDevice S112
ble_sched.c (SWI2)
pętla główna
alarm.c


Strona WWW
Przeglądarka i BLE OS
SoftDevice S112
ble_sched.c (SWI2)
pętla główna, sched_store.c
Zapis rozkładu
16 writeValue WithResponse (113 B)
alt
[MTU ≥ 116]
17 Write Request
18 RW_AUTHORIZE (WRITE_REQ)
19 validate
[MTU < 116]
20 Prepare Write ×n (bufor nrf_ble_qwr)
21 Execute Write
22 RW_AUTHORIZE (EXEC_WRITE)
23 qwr_cb: validate
24 authorize_reply (status)
validate = sched_decode;
OK → s_stage_ready
(długi: po odpowiedzi)
25 Write Response lub błąd ATT
26 table_replace, sched_store_save, limit 30 s
27 flash: kasowanie, zapis 0x7F000
28 readValue(), porównanie
Rozłączenie
29 limit lub alarm: disconnect
30 LL_TERMINATE
31 gattserver disconnected
32 GAP_EVT_ DISCONNECTED
bez połączenia: po 60 s
ADV_SET_TERMINATED
Strona WWW
Przeglądarka i BLE OS
SoftDevice S112
ble_sched.c (SWI2)
pętla główna, sched_store.c